

Serial Decode — Keithley DMM6500, DAQ6510, DMM7510, SMU2461

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.25** · MIT licence (see [LICENSE](#))

SERIAL DECODE - 240B FRAME

End App

BAUD	FORMAT	IDLE	LOGIC	THRESH	SA/BIT	RATE	BYTES	ERR	FIT	S/N
9600	8N1	HIGH	3V3 CMOS	1.63 V	8.3	80 kS/s	239	0	1.00	74 dB
Start bit										
0000	8B 81 69 6E 20 63 75 6C					70 61 20 71 75 69 20 0D		??in culpa qui .		
0016	0A 6F 66 66 69 63 69 61					20 64 65 73 65 72 75 6E		.officia deserun		
0032	74 20 6D 6F 6C 6C 69 74					20 61 6E 69 6D 20 69 64		t mollit anim id		
0048	20 65 73 74 20 6C 61 62					6F 72 75 6D 2E 20 4C 6F		est laborum. Lo		
0064	72 65 6D 20 69 70 73 75					6D 20 64 6F 6C 6F 72 20		rem ipsum dolor		
0080	73 69 74 20 0D 0A 61 6D					65 74 2C 20 63 6F 6E 73		sit ..amet, cons		
0096	65 63 74 65 74 75 72 20					61 64 69 70 69 73 63 69		ectetur adipisci		
0112	6E 67 20 65 6C 69 74 2C					20 73 65 64 20 64 6F 20		ng elit, sed do		
0128	65 69 75 73 6D 6F 64 20					74 65 6D 70 6F 72 20 69		eiusmod tempor i		
0144	6E 63 69 64 69 64 75 6E					74 20 0D 0A 75 74 20 6C		ncididunt ..ut l		
0160	61 62 6F 72 65 20 65 74					20 64 6F 6C 6F 72 65 20		abore et dolore		
0176	6D 61 67 6E 61 20 61 6C					69 71 75 61 2E 20 55 74		magna aliqua. Ut		
0192	20 65 6E 69 6D 20 61 64					20 6D 69 6E 69 6D 20 76		enim ad minim v		
0208	65 6E 69 61 6D 2C 20 71					75 69 73 20 0D 0A 6E 6F		eniam, quis ..no		
0224	73 74 72 75 64 20 65 78					65 72 63 69 74 61 74		strud exercitat		
FRAME HEX pg 1/1 bytes 1-239/239 win 240 [done]						log: bytes053 719 B				
CaptureViewModeNewLogSaveOptions										

The instrument's own front panel, no host attached: 239 bytes of a 240-byte capture at 9600 baud 8N1, no framing errors, S/N 74 dB. The padlock on the BAUD cell means the rate is pinned here rather than re-detected each capture.

docs/MANUAL.md	using it: hooking up, the screen, the buttons, what it copes with, where it fails
docs/REFERENCE.md	every measured number, and the firmware limits behind them
docs/BENCH.md	the harness: the seeded sweep, the release gate stage by stage, reproducing a failure

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

What to know before trusting it

Tested on a DMM6500 and nothing else. Every measured number in these documents came off that one instrument. See [Which instruments](#).

Automatic rate detection has three known failures, and typing the rate in fixes all three. A short pattern repeated over and over can measure at a small multiple of its true rate; a logic low near **-2 V** can misdetect; above 115 200 baud a long run of one repeated byte can refuse. That is **8 points in 860** of a full bench plan for the first two and about 3 in a million for the third, and **none reaches an ordinary payload at a standard rate**: across 171 600 captures opened from 200 different points in the waveform, the fox, the 1 kB text payload, twelve random payloads and the walking-bit patterns are clean from 300 to 250 000 baud. All three are characterised in the [manual](#).

Flow control is verified electrically but never closed as a loop. One credit pulse per armed capture, measured on a scope, with nothing waiting for it at the other end. The width is not settable on this firmware, so a receiver needs an edge-triggered input rather than a polling loop.

Nothing stops a long job early. A touch press is not delivered while a script runs, and the front-panel TRIGGER key does **not** deliver `trigger.EVENT_DISPLAY` during a panel-initiated run, so a recording runs to its stated size. The key does still *gate* an armed capture under `Options ▶ Trigger = Trigger key`, firing the acquisition on the key edge in firmware.

The TRIGGER key cannot be made to start a capture, and that was tried rather than assumed. A touch button works because the firmware executes a Lua string attached to a display *object*; the key is not an object, so all it can set is a latch — and reading a latch needs Lua running, which is exactly what blocks the touch panel. Either the key works or the screen does.

One press is the whole interaction. A screenful is ~240 bytes; a recording files 8 192 or 32 768 bytes without stepping; beyond that, flow control makes the window a chunk size and credits the device until it stops sending. Ceilings and arithmetic are in the [manual](#).

It takes over the measurement function and gives it back. Capture selects **Digitize Voltage** on the 10 V range whatever the meter was measuring; the mode is read once at startup and restored by **End App**, on the range it was on. Two consequences, both deliberate: a function changed from the front panel while the app is loaded does not survive the exit, because the next Capture reselects the digitizer; and the soak in `bench/` bypasses this path, so a soak does leave the instrument digitizing.

Endurance, measured

Five unattended runs on real hardware, all five the instrument driving itself: it reads its plan from its own USB key, commands the generator over the LAN, decodes, judges and writes every cell back to the key.

No computer is attached to any of it. Per-run detail is in [docs/BENCH.md](#) under *Soak results*.

run	when	duration	cells judged	BAD	rate
the fortnight	09-12 → 09-24	282.2 h	202 281	1 917	0.95 %
the week	08-29 → 09-05	182.2 h	133 297	2 462	1.85 %
the 31-hour	09-11 → 09-12	31.7 h	22 636	232	1.03 %
the 17-hour	09-07	17.24 h	11 739	143	1.22 %
the 8-hour	09-08	8.28 h	6 708	76	1.13 %

The fortnight is the reference figure: 121 laps and 11.76 days, 0.948 % [0.906 – 0.991] 95 % CI, and nothing drifted across it — slope -0.0005 points per lap ($t = -0.34$), first 60 laps 0.950 % against last 60 laps 0.947 %, lap time spread 1.9 %. Stopped by the front-panel TRIGGER at L135C427, **0 events**

instigated, 0 cells unrecorded in 0 gaps, heap flat at 2 724 kB. Unlike the 31-hour run, which needed a 220-cell intermittent-contact window excluded from its raw 1.58 %, it has no window: 0.948 % is raw.

Those two are the only pair here that can be compared, and the fortnight shows no regression. Their decoders are functionally identical — the trees differ in comments, READMEs and the soak panel's run-naming and nothing else — so **$z = -1.13$ against the 31-hour run's 1.03 % means unchanged, not improved**, and no vector regresses (largest $z = +1.78$ against a +3.02 Bonferroni bar over 39 tests). The 17-hour and 8-hour runs predate the decode fast paths and the week predates more than that; neither is comparable. Two things the total hides: 163 BAD, 8.5 %, are the reciprocal-ratio gap between `sdec.snaptol` and the judge's 2 % rate test rather than a defect — all from five random rate draws, which also explains the only four laps that look like outliers — and the largest genuine mode is the documented **125000 → 250000 doubling** at 6.07 % of its cells, against 6.65 % before.

The week: 136 247 captures, twelve segments, not one instrument event. An event on this firmware is a modal box nothing in the app can suppress, so it is the thing that ends an unattended run; every segment closed with **0 events instigated, 0 of them from the panel, 0 cells unrecorded in 0 gaps**. The generator was the only part that needed help: **270 waveform re-selections, every one recovered** — 241 on the second attempt, 28 on the third, 1 on the fourth. `C1:ARWV?` answers with the *previous* waveform's name on roughly 4 of every 39 real switches, and re-sending the selection fixes it while re-reading does not; the 8-hour run reproduced that at 10.3 %. **The week's 1.85 % is not comparable with the other two** — an older build and a different matrix, laps of 1 326 and 1 333 cells with different vectors, amplitudes and waits — so the drop below it is not a measured improvement.

The two current-matrix runs pool: 219 BAD in 18 447 cells, 1.19 %, agreeing within 0.6 σ . Both are the 1 677-point matrix on the current build, each on one power cycle. Split by rate, the 8-hour run reads **1.28 %** at or below 115 200 Bd against **0.75 %** above it and the 17-hour run **1.34 %** and **0.89 %** — the high-rate band is the better half in both, there being more samples per bit to disagree about at the low end. The 8-hour run is the cleanest record on file: **0 no-decode, 0 inconclusive, 0 generator failures, 0 events instigated, 0 cells unrecorded**, ending `4 iteration(s) complete` under its own control.

Heap and lap time were flat. Read after collection at every lap seam: **2678, 2678, 2679, 2678, 2679, 2678, 2678 kB across 10 062 cells** in the 17-hour run, **2687, 2687, 2687 kB across 5 031 cells** in the 8-hour. Flat lap time catches a leak that no failure count can see.

A low single-digit failure rate is the expected shape, because the vectors are built to break things — impulse spikes stacking to 9.3 V on a 3.3 V line, summed noise and a second signal riding on the logic, 6 % and 10 % mid-waveform baud drift, 2/10/20 % jitter, an inverted line, LIN break fields no UART frame can legally contain, 512-byte runs of one value, payloads longer than the capture — and **every one is swept across all 43 rates**. Composition is what stops that being an excuse. Only the seven vectors built to break something are allowed to fail — `v47`, `v48a/b`, `j20`, `v61/62/63`, class `loud` in `tools/soakplan.py` — and they carry **39 % of the failures; the other 61 % are exact vectors, where a failure is a defect** and is counted as one (86 and 132 of 218 pooled; the week's split was 32/68). **A confidently wrong byte fails every class**, `loud` included: declining to answer is a pass, being wrong while claiming to be right never is. Refusals are concentrated and explicable — in the whole 8-hour run only `v48b` (168), `v48a` (158) and `v47` (32) refuse a cell at all, and the drift vectors' reason is `no clear logic levels`, the correct answer to a band that has moved onto its own threshold.

Confidently wrong bytes, across all three runs: none. Every failure is the app reporting a rate its own framing contradicts, or declining to answer. Over **1 066 400 offline decodes, 95.9 % of failures are the rate being wrong and 0.026 % get a byte wrong with the rate right** — see **Rate detection and decode, counted separately** in [REFERENCE.md](#).

Power cycle the DMM between the smoke gate and a soak. `acq: line is idle (no transitions)` is accumulated state in the instrument: a soak started shortly after the 13-minute smoke went 33 idle in its first 49 cells, with the generator reading back the correct waveform throughout. Power-cycling the generator changes nothing; power-cycling the DMM clears it. The cheap tell is cell *time* — an idle cell fails in about 1 s against a healthy 5.4–6.4 s, so a poisoned run visibly races — and it costs a bench run rather than correctness, an idle cell being a refusal and not a wrong answer.

The offline twin is no longer the safe side. Compared only at a matched stimulus, over **444 405 cells in 265 laps** it fails on **1.135 %** against the bench's **1.187 %**: 0.96×, 0.67 σ from parity. Earlier readings of "over-predicting by 1.48×" compared a stressed harness against an unstressed bench and are withdrawn. That parity is partly two errors cancelling — the twin runs four vectors about 1.3× hot and invents ~1.5 failures a lap on random payloads the bench never fails, while scoring zero on the two drift vectors, where it is the *harsher* side, declining 100 % of those cells at the acquisition gate against the bench's 92–98 %. Worth knowing before leaning on it as a gate by itself; the per-vector table is in [BENCH.md](#).

Which instruments

The app needs a digitizer reachable as `dmm digitize` or `smu digitize`, the **touchscreen app API** (`display.create` and friends), and **Lua 5.0.2**. Every digitizer below runs at **1 MS/s**, and the **panel is pixel-identical across the whole TSP range**, so the app's screens carry over as they are.

DMM6500	Tested , firmware 1.7.17a — a 7.6-day run of 136 247 captures with no computer attached and zero instrument events, then 17.24-hour and 8.28-hour runs on the current matrix, and the namespace resolver verified here. 16-bit digitizer, " <i>maximum resolution 16 bits</i> ", specifications, April 2018
DAQ6510	Should run unmodified, on the strongest grounds of any untested model: it shares the UI board and the acquisition board with the DMM6500, only the channel-board plugin differing. Untested
DMM7510	Should run unmodified: 18-bit digitizer, better acquisition boards. Untested
SMU2461	Will install and try; may or may not work. Dual 18-bit digitizers, reached as <code>smu digitize</code> by a namespace the app resolves at load. That mechanism is verified on the DMM6500; no SMU has ever run it. Three unknowns below
2470 SMU	No. " <i>Digitized measurements are not a feature on the 2470</i> " — its reference manual, rev D, October 2024
2450 / 2460 SMU	Almost certainly not, by absence rather than denial: neither claims a digitizer, and the 2450 reference (rev D, May 2015) documents <code>smu.measure.*</code> with no digitize function

Porting is an acquisition question only. The app resolves `dmm digitize` or `smu digitize` once at load, with no test on any capture path, and the display layer carries over untouched. Timing transfers, because every rate figure in REFERENCE descends from the sample rate and the reading buffer's throughput; resolution is the one axis the app is indifferent to, thresholding each sample into a one or a zero. What to re-measure is what goes *through the acquisition board* — the tolerance envelope, the level and offset limits, the −2 V band. The DAQ6510 shares that board, so a deviation there is a real finding; the DMM7510's differs and those figures may move either way.

On an SMU2461, three things could stop it. A 2461 digitizes voltage and current **simultaneously** while the decoder trusts consecutive indices, so a buffer carrying current or source values alongside voltage reads as a waveform and surfaces as garbage bytes rather than an error. The app pins **10 V**, chosen for the

widest digitize bandwidth, which a 2461 may not offer. And it sets the source output off before every capture and **reads it back, refusing the capture if it cannot confirm it** — those constant names are unverified, and that refusal is the failure it is designed for.

The `.tspa` header decides where it installs, independently of whether the code would run: `$Product: DMM6500, DAQ6510, DMM7510, SMU2461`. Neither `DMM7510` nor `SMU2461` appears in the value set Keithley's app-header spec publishes, though Keithley ship TSP apps of their own declaring `DMM7510` — an install failure is the symptom if a firmware validates the field strictly, and this field is the first thing to revert.

If you run this on anything but a DMM6500, please say so — [open an issue](#). Every row above except the first is inference from API surfaces and vendor documents, and one report from real hardware outweighs all of it. Useful to include: `localnode.model` and `localnode.version`, whether **Manage Apps** offered the app at all, whether both screens built, and whether a capture decoded.

How to report bugs

[docs/BETA.md](#) (PDF) is the note to read first, and the one to hand to anybody trying this on their own bench: it lists the four behaviours that **look** like defects and are not — chiefly that a signal whose band straddles ground is decoded inverted, deliberately.

Then [open an issue](#) with, as far as you can: a **screen grab** of the whole 800x480 frame (`tools/screenshot.py` takes one over LXI with nothing loaded), the **text that should have been decoded**, the **bit rate** you set, and the **framing** you sent. The expected payload is the one piece that cannot be reconstructed from anything else — it turns "the decode looks wrong" into a difference that can be computed.

A wrong number that looks plausible is worth more than a crash.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order. [docs/BENCH.md](#) has every stage and what it checks.

```
python3 tools/release_sweep.py --offline # ~1 min, no instruments
python3 tools/bench_smoke.py             # 13 min, both instruments
                                         # then POWER CYCLE THE DMM -- see Endurance
above
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96,v97
python3 tools/release_sweep.py           # the whole thing, instruments included
```

Name all three skipped vectors, or the run is not the run you think it is. `soak.py`'s `--skip-vectors` defaults to empty and the skip is applied *before* the shuffle, so it sets the lap size and keys every amplitude, offset and wait: `v95,v96,v97` gives a **1 677**-cell lap, while naming only `v95,v96` gives 1 720 and lets `v97` back in, where it reads as a generator wedge — that is where an earlier run's 129 "generator failures" came from. `run_bench.py` already defaults to all three, so do not pass the flag to it at all.

No version is tagged without at least 8 hours of soak. A soak reports a failure rate per test point where a sweep reports one pass or one failure. A lap is 1 677 cells and about 2.1–2.5 hours, so 8 hours is the least that separates "fails every lap" from "failed once". Every lap draws its waveform order, its non-standard

rates and the wait before each capture from the lap number, so `--iteration N` rebuilds any lap exactly without running the ones before it, and a failure replays offline in seconds.

Code changes do not get pushed without passing the smoke gate. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/`; a `pre-push` hook recomputes it and refuses if either tree has moved. Documentation-only pushes are exempt, and when the bench is unavailable `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason. See [tools/hooks/README.md](#).

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Layout

tsp/	the app. <code>serial_core</code> acquisition, <code>uart_decode</code> framing, <code>chunk_decode</code> resumable decode, <code>serial_ui</code> panel, <code>serial_app</code> orchestration
bench/	the soak that runs on the instrument, loaded beside the app for a multi-day run and never shipped inside it. That README is the runbook
tools/	harnesses. <code>release_sweep.py</code> is the entry point; the rest are the authorities it calls
docs/	the manual, the measured reference, panel mockups, BETA.md for testers, and three firmware reports for Keithley

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Two instruments over LAN: the DMM6500 under test and an SDG2000X-series generator producing the stimulus. **A scope is optional** — an SDS1000X-E is used here to confirm the stimulus is what it claims to be, it only ever reads, and nothing in the gate needs it. `tools/instruments.py` holds the addresses and the hazards: repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.

Neither line's top model is needed. The bench asks the generator for 25 MSa/s of TrueArb, 20 Vpp and ~41 stored waveforms, and the scope, if used, for UART decode, two buses and 1 GSa/s. In both lines the model number is the *analog bandwidth* and the fastest edge here is a 250 kBd bit, so an **SDG2042X (40 MHz) and an SDS1104X-E (100 MHz) are sufficient**; the units here are an SDG2122X and an SDS1204X-E. `SDG_MAX_SRATE` is capped at 40 MSa/s so a plan cannot quietly outgrow the cheapest generator.

The generator must be 2000X-series or better, and an SDG1000X will NOT do — worth stating because the scope here is a 1000X-E and the numbers invite the wrong substitution. TrueArb is selected by the `SRATE` command, and Siglent's programming guide lists its availability per family:

	SDG800	SDG1000	SDG2000X	SDG5000	SDG1000X	SDG6000X/X-E	SDG7000A
<code>SRATE</code>	no	no	yes	no	no	yes	yes

An SDG1000X plays arbitrary waveforms DDS-only, and DDS **resamples** the stored points — the fall-back `SDG.assert_truearb()` checks for on every load. The vectors would stop being valid stimulus, because sub-sample edge timing is the quantity this project measures. `INTER`, the interpolation-method control, is

no even on the SDG2000X and reachable only on the 6000X/7000A, which is why a 2000X's TrueArb output is a plain held staircase with nothing to switch off.